AMRUTA INSTITUTE OF ENGINEERING AND MANAGEMENT SCIENCES – POLYTECHNIC

Course: Database System Concepts and PL/SQL

Course Code: 20CS34P

Week 10

The SQL comparison operators allow you to test if two expressions are the same. The following table illustrates the comparison operators in SQL:

Comparison Operator	Description
=	Equal
<>	Not Equal
!=	Not Equal
>	Greater Than
>=	Greater Than or Equal
<	Less Than
<=	Less Than or Equal
IN ( )	Matches a value in a list

Comparison Operator	Description
NOT	Negates a condition
BETWEEN	Within a range (inclusive)
IS NULL	NULL value
IS NOT NULL	Non-NULL value
LIKE	Pattern matching with % and _
EXISTS	Condition is met if subquery returns at least one row

Equality Operator

In SQL, you can use the = operator to test for equality in a query.

In this example, we have a table called suppliers with the following data:

supplier_id	supplier_name	city	state
100	Microsoft	Redmond	Washington
200	Google	Mountain View	California
300	Oracle	Redwood City	California
400	Kimberly-Clark	Irving	Texas

supplier_id	supplier_name	city	state
500	Tyson Foods	Springdale	Arkansas
600	SC Johnson	Racine	Wisconsin
700	Dole Food Company	Westlake Village	California
800	Flowers Foods	Thomasville	Georgia
900	Electronic Arts	Redwood City	California

Enter the following SQL statement:

```
SELECT *
FROM suppliers
WHERE supplier_name = 'Microsoft';
```

There will be 1 record selected. These are the results that you should see:

supplier_id	supplier_name	City	state
100	Microsoft	Redmond	Washington

Inequality Operator

In SQL, there are two ways to test for inequality in a query. You can use either the `<>` or `!=` operator. Both will return the same results.

Let's use the same suppliers table as the previous example.

supplier_id	supplier_name	City	state
100	Microsoft	Redmond	Washington
200	Google	Mountain View	California
300	Oracle	Redwood City	California
400	Kimberly-Clark	Irving	Texas
500	Tyson Foods	Springdale	Arkansas
600	SC Johnson	Racine	Wisconsin
700	Dole Food Company	Westlake Village	California
800	Flowers Foods	Thomasville	Georgia
900	Electronic Arts	Redwood City	California

Enter the following SQL statement to test for inequality using the <> operator:

```
SELECT *
FROM suppliers
WHERE supplier_name <> 'Microsoft';
```

OR enter this next SQL statement to use the != operator:

```
SELECT *
FROM suppliers
```

```
WHERE supplier_name != 'Microsoft';
```

There will be 8 records selected. These are the results you should see with either one of the SQL statements:

supplier_id	supplier_name	city	state
200	Google	Mountain View	California
300	Oracle	Redwood City	California
400	Kimberly-Clark	Irving	Texas
500	Tyson Foods	Springdale	Arkansas
600	SC Johnson	Racine	Wisconsin
700	Dole Food Company	Westlake Village	California
800	Flowers Foods	Thomasville	Georgia
900	Electronic Arts	Redwood City	California

Greater Than Operator

You can use the > operator in SQL to test for an expression greater than.

In this example, we have a table called *customers* with the following data:

customer_id	last_name	first_name	favorite_website
4000	Jackson	Joe	techonthenet.com

customer_id	last_name	first_name	favorite_website
5000	Smith	Jane	digminecraft.com
6000	Ferguson	Samantha	bigactivities.com
7000	Reynolds	Allen	checkyourmath.com
8000	Anderson	Paige	NULL
9000	Johnson	Derek	techonthenet.com

Enter the following SQL statement:

```
SELECT *
FROM customers
WHERE customer_id > 6000;
```

There will be 3 records selected. These are the results that you should see:

customer_id	last_name	first_name	favorite_website
7000	Reynolds	Allen	checkyourmath.com
8000	Anderson	Paige	NULL
9000	Johnson	Derek	techonthenet.com

Greater Than or Equal Operator

In SQL, you can use the >= operator to test for an expression greater than or equal to.

Let's use the same *customers* table as the previous example.

customer_id	last_name	first_name	favorite_website
4000	Jackson	Joe	techonthenet.com
5000	Smith	Jane	digminecraft.com
6000	Ferguson	Samantha	bigactivities.com
7000	Reynolds	Allen	checkyourmath.com
8000	Anderson	Paige	NULL
9000	Johnson	Derek	techonthenet.com

Enter the following SQL statement:

```
SELECT *

FROM customers

WHERE customer_id >= 6000;
```

There will be 4 records selected. These are the results that you should see:

customer_id	last_name	first_name	favorite_website
6000	Ferguson	Samantha	bigactivities.com
7000	Reynolds	Allen	checkyourmath.com

customer_id	last_name	first_name	favorite_website
8000	Anderson	Paige	NULL
9000	Johnson	Derek	techonthenet.com

Less Than Operator

You can use the < operator in SQL to test for an expression less than.

In this example, we have a table called *products* with the following data:

product_id	product_name	category_id
1	Pear	50
2	Banana	50
3	Orange	50
4	Apple	50
5	Bread	75
6	Sliced Ham	25
7	Kleenex	NULL

Enter the following SQL statement:

```
SELECT *

FROM products

WHERE product_id < 5;
```

There will be 4 records selected. These are the results that you should see:

product_id	product_name	category_id
1	Pear	50
2	Banana	50
3	Orange	50
4	Apple	50

Less Than or Equal Operator

In SQL, you can use the <= operator to test for an expression less than or equal to.

Let's use the same *products* table as the previous example.

product_id	product_name	category_id
1	Pear	50
2	Banana	50
3	Orange	50

product_id	product_name	category_id
4	Apple	50
5	Bread	75
6	Sliced Ham	25
7	Kleenex	NULL

Enter the following SQL statement:

```
SELECT *
FROM products
WHERE product_id <= 5;
```

There will be 5 records selected. These are the results that you should see:

product_id	product_name	category_id
1	Pear	50
2	Banana	50
3	Orange	50
4	Apple	50
5	Bread	75

IN operator

The SQL IN condition (sometimes called the IN operator) allows you to easily test if an expression matches any value in a list of values. It is used to help reduce the need for multiple OR conditions in a SELECT, INSERT, UPDATE, or DELETE statement.

The syntax for the IN condition in SQL is:

```
expression IN (value1, value2, .... value_n);
```

OR

```
expression IN (subquery);
```

where expression is a value to test and

value1, value2 ..., value_n are the values to test against *expression*. If any of these values matches *expression*, then the IN condition will evaluate to true.

Subquery is a SELECT statement whose result set will be tested against *expression*. If any of these values matches *expression*, then the IN condition will evaluate to true.

In this example, we have a table called *suppliers* with the following data:

supplier_id	supplier_name	city	state
100	Microsoft	Redmond	Washington
200	Google	Mountain View	California
300	Oracle	Redwood City	California
400	Kimberly-Clark	Irving	Texas
500	Tyson Foods	Springdale	Arkansas

supplier_id	supplier_name	city	state
600	SC Johnson	Racine	Wisconsin
700	Dole Food Company	Westlake Village	California
800	Flowers Foods	Thomasville	Georgia
900	Electronic Arts	Redwood City	California

Enter the following SQL statement:

```
SELECT *
FROM suppliers
WHERE supplier_name IN ('Microsoft', 'Oracle', 'Flowers Foods');
```

There will be 3 records selected. These are the results that you should see:

supplier_id	supplier_name	city	state
100	Microsoft	Redmond	Washington
300	Oracle	Redwood City	California
800	Flowers Foods	Thomasville	Georgia

NOT Operator

The SQL NOT condition (sometimes called the NOT Operator) is used to negate a condition in the WHERE clause of a SELECT, INSERT, UPDATE, or DELETE statement.

The syntax for the NOT condition in SQL is:

NOT condition

Where condition is the condition to negate. The opposite of the *condition* must be met for the record to be included in the result set.

In this example, we have a table called *products* with the following data:

product_id	product_name	category_id
1	Pear	50
2	Banana	50
3	Orange	50
4	Apple	50
5	Bread	75
6	Sliced Ham	25
7	Kleenex	NULL

Enter the following SQL statement:

```
SELECT *

FROM products

WHERE product_name NOT IN ('Pear', 'Banana', 'Bread');
```

There will be 4 records selected. These are the results that you should see:

product_id	product_name	category_id
3	Orange	50
4	Apple	50
6	Sliced Ham	25
7	Kleenex	NULL

This example would return all rows from the *products* table where the product_name is not Pear, Banana or Bread.

BETWEEN

The SQL BETWEEN condition allows you to easily test if an expression is within a range of values (inclusive). It can be used in a SELECT, INSERT, UPDATE, or DELETE statement.

The syntax for the BETWEEN condition in SQL is:

```
expression BETWEEN value1 AND value2;
```

where expression is a column or calculation.

value1 and value2 create an inclusive range that *expression* is compared to.

In this example, we have a table called *suppliers* with the following data:

supplier_id	supplier_name	city	state
100	Microsoft	Redmond	Washington
200	Google	Mountain View	California

supplier_id	supplier_name	city	state
300	Oracle	Redwood City	California
400	Kimberly-Clark	Irving	Texas
500	Tyson Foods	Springdale	Arkansas
600	SC Johnson	Racine	Wisconsin
700	Dole Food Company	Westlake Village	California
800	Flowers Foods	Thomasville	Georgia
900	Electronic Arts	Redwood City	California

Enter the following SELECT statement:

```
SELECT *
FROM suppliers
WHERE supplier_id BETWEEN 300 AND 600;
```

There will be 4 records selected. These are the results that you should see:

supplier_id	supplier_name	city	state
300	Oracle	Redwood City	California
400	Kimberly-Clark	Irving	Texas
500	Tyson Foods	Springdale	Arkansas

supplier_id	supplier_name	city	state
600	SC Johnson	Racine	Wisconsin

This example would return all rows from the *suppliers* table where the *supplier_id* is between 300 and 600 (inclusive).

IS NULL

The IS NULL condition is used in SQL to test for a NULL value. It returns TRUE if a NULL value is found, otherwise it returns FALSE. It can be used in a SELECT, INSERT, UPDATE, or DELETE statement.

The syntax for the IS NULL condition in SQL is:

```
expression IS NULL
```

where expression is the expression to test for a NULL value.

In this example, we have a table called *customers* with the following data:

customer_id	last_name	first_name	favorite_website
4000	Jackson	Joe	techonthenet.com
5000	Smith	Jane	digminecraft.com
6000	Ferguson	Samantha	bigactivities.com
7000	Reynolds	Allen	checkyourmath.com
8000	Anderson	Paige	NULL
9000	Johnson	Derek	techonthenet.com

Enter the following SQL statement:

```
SELECT *
FROM customers
WHERE favorite_website IS NULL;
```

There will be 1 record selected.

customer_id	last_name	first_name	favorite_website
8000	Anderson	Paige	NULL

This example will return all records from the *customers* table where the *favorite_website* contains a NULL value.

IS NOT NULL

The IS NOT NULL condition is used in SQL to test for a non-NULL value. It returns TRUE if a non-NULL value is found, otherwise it returns FALSE. It can be used in a SELECT, INSERT, UPDATE, or DELETE statement.

The syntax for the IS NOT NULL condition in SQL is:

```
expression IS NOT NULL
```

where the expression is to test for a NOT NULL value.

In this example, we have a table called products with the following data:

product_id	product_name	category_id
1	Pear	50
2	Banana	50

product_id	product_name	category_id
3	Orange	50
4	Apple	50
5	Bread	75
6	Sliced Ham	25
7	Kleenex	NULL

Enter the following SQL statement:

```
SELECT *
FROM products
WHERE category_id IS NOT NULL;
```

There will be 6 records selected. These are the results that you should see:

product_id	product_name	category_id
1	Pear	50
2	Banana	50
3	Orange	50
4	Apple	50
5	Bread	75

product_id	product_name	category_id
6	Sliced Ham	25

This example will return all records from the *products* table where the *customer_id* does not contain a NULL value.

LIKE

The SQL LIKE condition allows you to use wildcards to perform pattern matching in a query. The LIKE condition is used in the WHERE clause of a SELECT, INSERT, UPDATE, or DELETE statement.

The syntax for the LIKE condition in SQL is:

```
expression LIKE pattern
```

where expression is a character expression such as a column or field and

pattern is a character expression that contains pattern matching. The wildcards that you can choose from are:

Wildcard	Explanation
%	Allows you to match any string of any length (including zero length)
_	Allows you to match on a single character

In this example, we have a table called *customers* with the following data:

customer_id	last_name	first_name	favorite_website
4000	Jackson	Joe	techonthenet.com

customer_id	last_name	first_name	favorite_website
5000	Smith	Jane	digminecraft.com
6000	Ferguson	Samantha	bigactivities.com
7000	Reynolds	Allen	checkyourmath.com
8000	Anderson	Paige	NULL
9000	Johnson	Derek	techonthenet.com

Enter the following SQL statement:

```
SELECT *
FROM customers
WHERE last_name LIKE 'J%'
ORDER BY last_name;
```

There will be 2 records selected. These are the results that you should see:

customer_id	last_name	first_name	favorite_website
4000	Jackson	Joe	techonthenet.com
9000	Johnson	Derek	techonthenet.com

This example returns the records in the customers table where the last_name starts with 'J'. As you can see, the records for the last names Jackson and Johnson have been returned.

Using Multiple % Wildcards in the LIKE Condition

You can also using the % wildcard multiple times with the LIKE condition.

Using the same *customers* table with the following data:

customer_id	last_name	first_name	favorite_website
4000	Jackson	Joe	techonthenet.com
5000	Smith	Jane	digminecraft.com
6000	Ferguson	Samantha	bigactivities.com
7000	Reynolds	Allen	checkyourmath.com
8000	Anderson	Paige	NULL
9000	Johnson	Derek	techonthenet.com

Let's try to find all *last_name* values from the *customers* table where the *last_name* contains the letter 'e'. Enter the following SQL statement:

```
SELECT last_name

FROM customers

WHERE last_name LIKE '%e%'

ORDER BY last_name;
```

There will be 3 records selected. These are the results that you should see:

last_name
Anderson

last_name
Ferguson
Reynolds

In this example, the last names Anderson, Ferguson and Reynolds contain the letter 'e'.

Example - Using _ Wildcard in the LIKE Condition

_ wildcard is looking for exactly one character, unlike the % wildcard.

Using the *categories* table with the following data:

category_id	category_name
25	Deli
50	Produce
75	Bakery
100	General Merchandise
125	Technology

Let's try to find all records from the *categories* table where the *category_id* is 2-digits long and ends with '5'. Enter the following SQL statement:

```
SELECT *
FROM categories
WHERE category_id LIKE '_5';
```

There will be 2 records selected. These are the results that you should see:

category_id	category_name
25	Deli
75	Bakery

In this example, there are 2 records that will pattern match - the *category_id values* 25 and 75. Notice that the *category_id* of 125 was not selected because, the _ wilcard matches only on a single character.

Using Multiple _ Wildcards in the LIKE Condition

If you wanted to match on a 3-digit value that ended with '5', you would need to use the _ wildcard two times. You could modify your query as follows:

```
SELECT *
FROM categories
WHERE category_id LIKE '__5';
```

Now you will return the *category_id* value of 125:

category_id	category_name
125	Technology

<u>EXISTS</u>

The SQL EXISTS condition is used in combination with a subquery and is considered to be met, if the subquery returns at least one row. It can be used in a SELECT, INSERT, UPDATE, or DELETE statement.

The syntax for the EXISTS condition in SQL is:

```
WHERE EXISTS ( subquery );
```

Where *subquery* is a SELECT statement. If the *subquery* returns at least one record in its result set, the EXISTS clause will evaluate to true and the EXISTS condition will be met. If the *subquery* does not return any records, the EXISTS clause will evaluate to false and the EXISTS condition will not be met.

In this example, we have a *customers* table with the following data:

customer_id	last_name	first_name	favorite_website
4000	Jackson	Joe	techonthenet.com
5000	Smith	Jane	digminecraft.com
6000	Ferguson	Samantha	bigactivities.com
7000	Reynolds	Allen	checkyourmath.com
8000	Anderson	Paige	NULL
9000	Johnson	Derek	techonthenet.com

And a table called *orders* with the following data:

order_id	customer_id	order_date
1	7000	2016/04/18
2	5000	2016/04/18
3	8000	2016/04/19

order_id	customer_id	order_date
4	4000	2016/04/20

Now let's find all of the records from the *customers* table where there is at least one record in the *orders* table with the same *customer_id*. Enter the following SELECT statement:

```
SELECT *
FROM customers
WHERE EXISTS
  (SELECT *
   FROM orders
   WHERE customers.customer_id = orders.customer_id);
```

There will be 4 records selected. These are the results that you should see:

customer_id	last_name	first_name	favorite_website
4000	Jackson	Joe	techonthenet.com
5000	Smith	Jane	digminecraft.com
7000	Reynolds	Allen	checkyourmath.com
8000	Anderson	Paige	NULL

In this example, there are 4 records in the *customers* where the *customer_id* value appears in the *orders* table.

SubQueries

- A Subquery or Nested query is a query within another SQL query and embedded within the WHERE clause. A subquery is a query within a query

- A subquery is used to return data that will be used in the main query as a condition to further restrict the data to be retrieved.

- (3)Subqueries can be used with the SELECT, INSERT, UPDATE, and DELETE statements along with the operators like =, <, >, >=, <=, IN, BETWEEN etc.

- We need to put sub queries in parenthesis always

- We need to place subqueries on the right side of the comparison operator

- Use single row operator with single row subqueries and multiple row operator with multiple row subqueries

General Syntax

```
SELECT col1, col2
FROM   table1
WHERE  col1  OPERATOR
(SELECT col1
FROM table2
[WHERE])
```

Single Row Subquery

- It returns only one row of results and uses a single row operator (most common is the equal operator (=)). The other operators are > ,< ,>= ,=<

- Single row subqueries can select data from the same table or from another table

```
SELECT ENAME

FROM EMP

WHERE  SAL = (SELECT MIN(SAL) FROM EMP);
```

```
SQL> select * from emp;

     EMPNO ENAME      JOB              MGR HIREDATE         SAL       COMM
---------- ---------- --------- ---------- --------- ---------- ----------
    DEPTNO
----------
      7698 BLAKE      MANAGER         7839 01-MAY-07       2850
        10

      7782 CLARK      MANAGER         7839 09-JUN-08       2450
        10

      7788 SCOTT      ANALYST         7566 09-JUN-12       3000
        20

      7789 TPM        ANALYST         7566 09-JUN-17       3000

      7790 TOM        ANALYST         7567 09-JUL-17       4000


SQL> select * from dept;

    DEPTNO DNAME          LOC
---------- -------------- -------------
        10 ACCOUNTING     NEW YORK
        20 RESEARCH       DALLAS
        30 RETAIL         DELHI
        40 SALES          MUMBAI
```

```
SELECT ENAME

FROM EMP

WHERE DEPTNO =  (SELECT DEPTNO FROM DEPT WHERE DNAME = 'RESEARCH');
```

```
SQL> SELECT ENAME FROM EMP WHERE SAL = (SELECT MIN(SAL) FROM EMP);

ENAME
----------
CLARK

SQL> SELECT ENAME FROM EMP WHERE DEPTNO = (SELECT DEPTNO FROM DEPT WHERE DNAME =
 'RESEARCH');

ENAME
----------
SCOTT

SQL>
```

Multiple Row Subquery

- It returns several rows of results from the subquery, uses the IN operator.

- The other operator which are used is any and all

SELECT ENAME, DEPTNO FROM EMP

WHERE DEPTNO **IN** (SELECT DEPTNO FROM DEPT WHERE DNAME LIKE 'R%');

```
SQL> SELECT DEPTNO FROM DEPT WHERE DNAME LIKE 'R%';

    DEPTNO
----------
        20
        30
```

```
SQL> select ename ,deptno from emp where deptno in (select deptno from dept wher
e dname like 'R%');

ENAME          DEPTNO
---------- ----------
SCOTT              20
```

Correlated Subquery

- A correlated SQL subquery is just a subquery that is executed many times—once for each record (row) returned by the outer (main) query.
- In other words, the outer query returns a table with multiple rows; the inner query then runs once for each of those rows.
- If your outer query returns 10 rows, then the inner query will run 10 times. And if your outer query returns 100 rows, the inner query will run 100 times.

- So a **correlated subquery** (also known as a **synchronized subquery**) is a subquery (a query nested inside another query) that uses values from the outer query.
- Because the subquery may be evaluated once for each row processed by the outer query, it can be slow.

Here is an example for a typical correlated subquery. In this example, the objective is to find all employees whose salary is above average for their department.

```
SELECT employee_number, name
FROM employees emp
WHERE salary > (    SELECT AVG(salary)
                    FROM employees
                    WHERE department = emp.department );
```

In the above query the outer query is

```
SELECT employee_number, name
  FROM employees emp
  WHERE salary > ...
```

and the inner query (the correlated subquery) is

```
SELECT AVG(salary)
  FROM employees
  WHERE department = emp.department
```

In the above nested query the inner query has to be re-executed for each employee

Correlated subquery with EXISTS operator

- The `EXISTS` operator is used to test for the existence of any record in a subquery.
- The `EXISTS` operator returns TRUE if the subquery returns one or more records.

EXISTS Syntax

```
SELECT column_name(s)
FROM table_name
WHERE EXISTS   (SELECT column_name
                FROM table_name
                WHERE condition);
```

The EXISTS operator is used to check the existence of a result of a subquery. Using EXISTS the following query display the employee_id, manager_id, first_name and last_name of those employees who manage other employees.

```
SELECT employee_id, manager_id, first_name, last_name

FROM employees a

WHERE EXISTS (SELECT employee_id

              FROM employees b

              WHERE b.manager_id = a.employee_id)
```

Example 2: we have a *customers* table with the following data:

customer_id	last_name	first_name	favorite_website
4000	Jackson	Joe	techonthenet.com
5000	Smith	Jane	digminecraft.com
6000	Ferguson	Samantha	bigactivities.com
7000	Reynolds	Allen	checkyourmath.com

customer_id	last_name	first_name	favorite_website
8000	Anderson	Paige	NULL
9000	Johnson	Derek	techonthenet.com

And a table called *orders* with the following data:

order_id	customer_id	order_date
1	7000	2016/04/18
2	5000	2016/04/18
3	8000	2016/04/19
4	4000	2016/04/20

Now let's find all of the records from the customers table where there is at least one record in the orders table with the same customer_id. Enter the following SELECT statement:

```
SELECT *
FROM customers
WHERE EXISTS
  (SELECT *
   FROM orders
   WHERE customers.customer_id = orders.customer_id);
```

There will be 4 records selected. These are the results that you should see:

customer_id	last_name	first_name	favorite_website
4000	Jackson	Joe	techonthenet.com
5000	Smith	Jane	digminecraft.com
7000	Reynolds	Allen	checkyourmath.com
8000	Anderson	Paige	NULL

In this example, there are 4 records in the customers where the customer_id value appears in the orders table.

Correlated subquery with NOT EXISTS operator

- NOT EXISTS is logically opposite of EXISTS operator.
- NOT EXISTS is used when we need to check if rows do not exist in the results returned by a subquery.

NOT EXISTS Syntax

```
SELECT column_name(s)
FROM table_name
WHERE NOT  EXISTS   (SELECT column_name
                     FROM table_name
                     WHERE condition);
```

Using NOT EXISTS the following query display the employee_id, manager_id, first_name and last_name of those employees who have no manager status. This query is opposite to the previous one.

```
SELECT employee_id, manager_id, first_name, last_name

FROM employees a

WHERE NOT EXISTS (SELECT employee_id

                  FROM employees b

                  WHERE b.manager_id = a.employee_id);
```

For example, the following query returns all employees who have no dependents:

```
SELECT    employee_id,    first_name,    last_name
FROM    employees e
WHERE    NOT EXISTS( SELECT    *
                     FROM  dependents d
                     WHERE  d.employee_id = e.employee_id)
                     ORDER BY first_name , last_name;
```

The following picture shows the output:

employee_id	first_name	last_name
121	Adam	Fripp
193	Britney	Everett
179	Charles	Johnson
126	Irene	Mikkilineni
177	Jack	Livingston
178	Kimberely	Grant
120	Matthew	Weiss
122	Payam	Kaufling
192	Sarah	Bell
123	Shanta	Vollman

Example 3:

In this example, we have a table called *customers* with the following data:

customer_id	last_name	first_name	favorite_website
4000	Jackson	Joe	techonthenet.com
5000	Smith	Jane	digminecraft.com
6000	Ferguson	Samantha	bigactivities.com
7000	Reynolds	Allen	checkyourmath.com
8000	Anderson	Paige	NULL
9000	Johnson	Derek	techonthenet.com

And a table called *orders* with the following data:

order_id	customer_id	order_date
1	7000	2016/04/18
2	5000	2016/04/18
3	8000	2016/04/19
4	4000	2016/04/20
5	NULL	2016/05/01

Enter the following SQL statement:

```
SELECT *
FROM customers
WHERE NOT EXISTS
  (SELECT *
   FROM orders
   WHERE customers.customer_id = orders.customer_id);
```

There will be 2 records selected.

customer_id	last_name	first_name	favorite_website
6000	Ferguson	Samantha	bigactivities.com
9000	Johnson	Derek	techonthenet.com

This example would return all records from the *customers* table where there are **no** records in the *orders* table for the given *customer_id*.

Correlated subquery with ANY operator

The ANY operator is used to compare a value to a list of values or result set returned by a subquery. The following illustrates the syntax of the ANY operator when it is used with a list or subquery:

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator ANY   (SELECT column_name
                                  FROM table_name
                                  WHERE condition);
```

In this syntax:

- The ANY operator must be preceded by a comparison operator such as =, !=, >, >=,<, <=.
- The list or subquery must be surrounded by the parentheses.
- ANY means that the condition will be true if the operation is true for any of the values in the range.

For example,consider the tables example1 and example2,

id	name
1	suresh
2	dasari
3	rohini
4	madhav
5	honey

example1

© Tutlane.com

id	name
1	suresh
4	madhav
5	honey

example2

```
SELECT id,name
FROM example1
WHERE id = ANY(SELECT Id  FROM example2)
```

When we execute the above SQL query, we will get a SQL any operator result like as shown below.

	id	name
1	1	suresh
2	4	madhav
3	5	honey

© Tutlane.com

```
(3 row(s) affected)
```

```
SELECT id,name
FROM example1
id >= ANY(SELECT Id FROM example2)
```

When we execute the above query, we will get the result as shown below.

	id	name
1	1	suresh
2	2	dasari
3	3	rohini
4	4	madhav
5	5	honey

© Tutlane.com

(5 row(s) affected)

For example, the following statement returns all products whose list price is greater than any list price of products in the category 1:

```
SELECT    product_name,    list_price
FROM    products
WHERE    list_price > ANY(      SELECT          list_price
                                FROM            products
                                WHERE           category_id = 1    )
```

Correlated subquery with ALL operator

ALL operator is used to compare a value to a list of values or result set returned by a subquery.

The following shows the syntax of the ALL operator used with a list or a subquery:

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator ALL   (SELECT column_name
                                  FROM table_name
                                  WHERE condition);
```

In this syntax:

- The ALL operator must be preceded by an comparison operator such as =, != >,>=, <, <= and followed by a subquery.
- The list or subquery must be surrounded by the parentheses.

- ALL means that the condition will be true only if the operation is true for all values in the range.

Consider the following Products Table and OrderDetails Table,

Products Table

ProductID	ProductName	SupplierID	CotegoryID	Price
1	Chais	1	1	18
2	Chang	1	1	19
3	Aniseed Syrup	1	2	10
4	Chef Anton's Cajun Seasoning	2	2	22
5	Chef Anton's Gumbo Mix	2	2	21
6	Boysenberry Spread	3	2	25
7	Organic Dried Pears	3	7	30
8	Northwoods Cranberry Sauce	3	2	40
9	Mishi Kobe Niku	4	6	97

OrderDetails Table

OrderDetailsID	OrderID	ProductID	Quantity
1	10248	1	12
2	10248	2	10
3	10248	3	15
4	10249	1	8
5	10249	4	4
6	10249	5	6
7	10250	3	5
8	10250	4	18
9	10251	5	2
10	10251	6	8
11	10252	7	9
12	10252	8	9
13	10250	9	20
14	10249	9	4

The followinq sql statement finds the name of the product if all the records in the OrderDetails has Quantity either equal to 6 or 2.

```
SELECT ProductName

FROM Products

WHERE ProductID = ALL (SELECT ProductId

                       FROM OrderDetails
```

```
WHERE Quantity = 6 OR Quantity = 2);
```

Output:

ProductName
Chef Anton's Gumbo Mix

Another example is the following query, which returns the names of employees whose salary is greater than the salary of all the employees in department 5:

SELECT Lname, Fname
FROM EMPLOYEE
WHERE Salary > **ALL** (**SELECT** Salary
FROM EMPLOYEE
WHERE Dno=5);

Example 2: the following statement returns all products whose list price is greater than all list price of products in the category 1:

```
SELECT product_name,list_price
FROM products
WHERE list_price > ALL    ( SELECT list_price
                            FROM products
                            WHERE category_id = 1 )
                            ORDER BY product_name;
```

Correlated subquery with SOME operator

- In SQL, the **SOME** operator is used to compare a value with a single column set of values returned by the subquery.
- The **SOME** operator in SQL must match at least one value in a subquery and that value must be preceded by comparison operators.

- Generally we will use this **SOME** operator in **WHERE** clause to check whether the required column values are matching with the set of values returned by subquery or not.

Syntax of SQL SOME Operator

```
SELECT column1,column2
FROM tablename
WHERE column1 = SOME(SELECT column1
                     FROM tablename
                     WHERE column1 ='somevalue')
```

If you observe above SQL **SOME** operator syntax, we will get values only when **column1** values matches with **column1** data returned by subquery otherwise it will not return any data.

empid	empname	designation	salary	Location
1	suresh	software engineer	25000	chennai
2	rohini	AEO	15000	chennai
3	madhavsai	business analyst	50000	nagpur
4	mahendra	CA	75000	guntur
5	sateesh	Doctor	65000	guntur

© Tutlane.com

The following sql statement will return employee details whose **salary** column values match with the data returned by the subquery.

```
SELECT *

FROM EmployeeDetails

WHERE salary = SOME(SELECT salary

                    FROM EmployeeDetails

                    WHERE salary >25000)
```

When we execute above sql query, we will get the result will be like as shown below.

empid	empname	designation	salary	Location
3	madhavsai	business analyst	50000	nagpur
4	mahendra	CA	75000	guntur
5	sateesh	Doctor	65000	guntur

© Tutlane.com

The following sql statement will return employee details whose **salary** column value matches with at least one value with the data returned by subquery and that value must be preceded by comparison operators.

```
SELECT *
FROM EmployeeDetails
WHERE salary > SOME(SELECT salary
                    FROM EmployeeDetails
                    WHERE salary >25000)
```

When we execute the above query, we will get the result as shown below.

empid	empname	designation	salary	Location
4	mahendra	CA	75000	guntur
5	sateesh	Doctor	65000	guntur

© Tutlane.com